

Game 02 — Tic Tac Toe (Complete Plan)

Game 2 of 7 Complexity: ★ Low Estimate: 1 day Status: Half built

Complete plan (supersedes the 2026-09-09 sample). Status: **built** — P0–P2 complete plus misère from P3; X/O theme picker skipped. `apps/frontend/public/games/tic-tac-toe/` (`index.html`, `style.css`, ESM `game.js`, `game.test.html`); jest twin at `src/__tests__/games-tic-tac-toe.test.ts` incl. the exhaustive minimax sweep. Constants below are the spec of record; §9–§11 list the only remaining work.

1. Overview

Classic tic tac toe: two players on one device, or single player vs. the computer at three AI levels. Hook is the running series score ("first to 5?") and a provably unbeatable Hard AI.

2. Complete game spec

- 3×3 board; **X moves first in round 1**; alternate marks each round; the **loser of a round starts the next**; after a draw, the alternating starter flips.
- Win = 3 in a row (8 lines: 3 rows, 3 cols, 2 diagonals). Draw = full board, no win.
- **Modes:** `2P local` (pass-and-play) · `1P` vs computer with levels:
 - **Easy** — uniformly random legal move.
 - **Medium** — win/block heuristic:
 1. take a winning move if one exists
 2. block an opponent win
 3. otherwise random legal move
 - **Hard** — full minimax (unbeatable); ties broken by first-found move in scan order (deterministic for tests).
- **Misère variant** (toggle): completing 3-in-a-row **loses**. Win detection is the same `checkWinner`; the `_interpretation_` flips (winner of a line loses the round). Draw rules unchanged.
- Illegal input impossible: occupied cell, finished round, or out-of-turn tap are no-ops; input locked during AI "thinking" tick (~300 ms artificial delay for feel).

3. Series model

Series = per-mode running tally. Persisted per `(mode, level)` key; Reset zeroes the current series only.

4. Screens & UI

State	Elements
menu	Mode pick (1P/2P), level pick (1P), misère toggle, series scoreboard, ▶ Play
playing	Board grid, turn indicator (mark + color), series mini-score, △ Menu
roundEnd	Overlay: "X wins! 🏆" / "O wins!" / "Draw 🟡" (misère wording flips), winning line
	highlighted through the 3 marks, [Next round] [Menu]

Board = CSS grid; marks render as inline SVG strokes with draw-in animation. Winning line gets a distinct highlight class. Fits 360 px viewport with no scroll.

5. Data model (localStorage)

```
game:tic-tac-toe:series:<modeKey> → { x: 3, o: 2, d: 1 } # modeKey = '2p' | '1p-easy' | '1p-medium' | '1p-hard'
game:tic-tac-toe:prefs              → { mode, level, misere, muted }
game:tic-tac-toe:best               → n/a (no score; share uses the series line)
```

6. File structure & function inventory

```
tic-tac-toe/
  index.html      # menu / playing / roundEnd sections
  style.css       # grid, marks, win highlight, overlays
  game.js         # ESM: pure logic exported + DOM wiring
  game.test.html  # browser twin of the jest suite
```

Pure exported logic: `newBoard()`, `applyMove(board, i, mark)`, `checkWinner(board) → { winner, line } | 'draw' | null`, `legalMoves(board)`, `minimaxScore(board, mark)`, `bestMove(board, mark)` (hard), `mediumMove(board, mark)`, `easyMove(board, mark, rng)`, `resolveMisere(result)`. DOM layer: screens, `renderBoard/renderCell`, `highlightWin(line)`, `isAiTurnNow()`, series storage, share.

7. Edge cases

- occupied / finished / out-of-turn taps = no-ops; double-tap debounce 50 ms.
- AI never acts during `roundEnd`; Next-round resets board but not series.
- Misère + AI: Hard minimax optimizes the `_misère_` objective (inverted leaf scores) — covered by the exhaustive test at this variant too.
- Storage disabled → defaults, no persistence, no crash.
- Series key changes when mode/level changes (no cross-mode pollution).
- Draw starter alternation persists only within a session (not in storage) — stated.

8. Testing plan (jest twin + game.test.html)

- `checkWinner`: all 8 lines, draw, empty, mid-game null.
- Hard AI: exhaustive sweep — never loses from any reachable position (both normal and misère objectives).
- Medium: always takes an available win; always blocks an immediate threat.
- Series persistence round-trip; mode-key isolation.
- Misère interpretation flips round outcome without changing `checkWinner`.

9. Task breakdown

P0/P1/P2 — DONE (2026-09-09/10 build pass)

- ☒ 2P local, win/draw detection, win-line highlight, round flow, series scoreboard
- ☒ 1P Easy/Medium/Hard (minimax), misère variant, SVG marks + animations
- ☒ Persistence, tests (incl. exhaustive unbeatable sweep), game.test.html

P2 — remaining

- ☐ **Resolve analytics coupling** — `game.js` (~lines 531–582) POSTs `game_played` with `?api=` base resolution. Under isolation: delete this block (owner call — see master README §7 conflict note).
- ☐ QA gate (master README §5): offline play + isolation greps.

P3 — remaining polish

- ☒ Misère variant — done
- ☒ Board flip animation — done
- ☐ X/O theme picker (skipped by owner decision — keep skipped unless requested)

10. Acceptance criteria

- §7 cases verified; §8 tests green in jest and browser twin.
- Hard AI unbeatable in exhaustive sweep (both objectives).
- Series survives reload; Reset works; 360 px fits with no scroll.

11. Deferred coupling (intentionally NOT built)

Online multiplayer (needs backend rooms), footer/nav links, Play Hub card, achievements — per master README §7. Existing analytics POST block is the one live violation; flagged above, not silently removed.